

**Departamento de Engenharia de Electrónica e Telecomunicações e de
Computadores**

**Resolução Automática de Sudoku:
Pesquisa em Espaço de Estados e Optimização**

51620 : Radesh Ilesh Gamanbhai Govind (51620@alunos.isel.pt)

51619 : Martim Silva Seriz Monteiro (51619@alunos.isel.pt)

Relatório para a Unidade Curricular de Introdução a Sistemas de Informação
da Licenciatura em Engenharia Informática e de Computadores

Professor : Professor Nuno Leite

Resumo

Este trabalho apresenta quatro solucionadores automáticos para o puzzle Sudoku clássico e para a sua variante Killer Sudoku. Em Prolog, foram implementados o algoritmo de aprofundamento iterativo (DFID) e o algoritmo A^* , ambos com a heurística de valores mínimos restantes (MRV). Em Python, foram implementados o arrefecimento simulado (SA) e o algoritmo genético (GA), com representação baseada em trocas intra-caixa. Todos os solucionadores foram estendidos para suportar Killer Sudoku. Os resultados mostram que o DFID com MRV resolve todas as instâncias em menos de quatro segundos. O A^* expande mais nós em puzzles com elevado grau de liberdade. O SA resolve de forma fiável instâncias fáceis e intermédias, ficando por vezes preso em mínimos locais de custo dois. O GA converge prematuramente e não encontra soluções completas dentro dos limites testados.

Abstract

This work presents four automatic solvers for the classic Sudoku puzzle and its Killer Sudoku variant. In Prolog, depth-first iterative deepening (DFID) and A* were implemented, both with the minimum remaining values (MRV) heuristic. In Python, simulated annealing (SA) and a genetic algorithm (GA) were implemented using a within-box swap representation. All solvers were extended to support Killer Sudoku cage constraints. Experimental results show that DFID with MRV solves all instances in under four seconds. A* expands more nodes on puzzles with high degrees of freedom. SA reliably solves easy and medium instances but occasionally gets trapped in local minima at cost two. The GA suffers from premature convergence and does not find complete solutions within the tested parameter limits.

Índice

Lista de Tabelas	ix
1 Introdução	1
1.1 Contexto	1
1.2 Objectivos	1
1.3 Estrutura do Relatório	2
2 Formulação do Problema e Implementação	3
2.1 Representação do Estado	3
2.1.1 Sudoku Clássico	3
2.1.2 Killer Sudoku	3
2.2 Pesquisa em Espaço de Estados — Prolog	4
2.2.1 Aprofundamento Iterativo (DFID)	4
2.2.2 Pesquisa A*	5
2.2.3 Extensão para Killer Sudoku — Prolog	5
2.3 Pesquisa em Espaço de Soluções — Python	6
2.3.1 Inicialização e Operador	6
2.3.2 Arrefecimento Simulado (SA)	6
2.3.3 Algoritmo Genético (GA)	7
2.3.4 Extensão para Killer Sudoku — Python	8
3 Resultados Experimentais e Conclusões	9
3.1 Configuração Experimental	9
3.2 Resultados dos Solucionadores Prolog	9
3.3 Resultados do Arrefecimento Simulado (SA)	10
3.4 Resultados do Algoritmo Genético (GA)	11
3.5 Análise Comparativa	11
3.6 Conclusões	12

Referências	13
A Estrutura do Repositório e Instruções de Execução	i
A.1 Estrutura de Ficheiros	i
A.2 Instruções de Execução	ii

Lista de Tabelas

3.1	Resultados do DFID e A* nos puzzles clássicos.	9
3.2	Resultados do DFID e A* no Killer Sudoku (fácil).	10
3.3	Resultados do SA (3 execuções por puzzle; limite: 300 000 iterações).	10
3.4	Resultados do GA (3 execuções; pop.=200, máx. 10 000 gerações, 15–26 s/execução). . . .	11
3.5	Comparação dos quatro solucionadores nos puzzles clássicos.	11

Capítulo 1

Introdução

1.1 Contexto

O Sudoku é um dos puzzles de lógica mais populares do mundo. O seu objectivo é simples: preencher uma grelha 9×9 com os dígitos de 1 a 9, de tal forma que cada linha, cada coluna e cada uma das nove sub-grelhas 3×3 (doravante denominadas caixas) contenha cada dígito exactamente uma vez. Matematicamente, este problema pertence à classe NP-completo [4], o que significa que não existe nenhum algoritmo eficiente geral para o resolver. Na prática, no entanto, a estrutura altamente restrita do puzzle permite que algoritmos especializados o resolvam rapidamente.

O Killer Sudoku é uma variante do Sudoku clássico. Para além das restrições de linhas, colunas e caixas, o Killer Sudoku introduz o conceito de gaiola (em inglês, cage): um grupo de células delimitado por uma linha pontuada, cujos valores devem somar um valor alvo e não se podem repetir dentro da mesma gaiola. Esta restrição adicional torna o espaço de busca simultaneamente mais complexo e mais restrito.

1.2 Objectivos

Este trabalho tem como objectivo o desenvolvimento de quatro solucionadores automáticos para o Sudoku clássico e para o Killer Sudoku, correspondendo a quatro paradigmas algorítmicos distintos:

1. **DFID com MRV** — aprofundamento iterativo com pesquisa em profundidade, implementado em Prolog, com a heurística de valores mínimos restantes (Minimum Remaining Values, MRV) para ordenação de variáveis;
2. **A*** — pesquisa heurística melhor-primeiro, implementado em Prolog, com uma heurística admissível baseada no número de células vazias;
3. **Arrefecimento Simulado (SA)** — algoritmo de optimização metaheurística, implementado em Python, com representação por trocas intra-caixa;
4. **Algoritmo Genético (GA)** — algoritmo evolutivo, implementado em Python, com cruzamento preservador de caixas e mutação por troca intra-caixa.

Todos os solucionadores foram estendidos para suportar a variante Killer Sudoku.

1.3 Estrutura do Relatório

O presente relatório está organizado da seguinte forma. O Capítulo 2 descreve a formulação do problema, a representação adoptada e a implementação detalhada de cada algoritmo. O Capítulo 3 apresenta os resultados experimentais e a análise comparativa dos quatro solucionadores, culminando nas conclusões do trabalho.

Capítulo 2

Formulação do Problema e Implementação

2.1 Representação do Estado

2.1.1 Sudoku Clássico

A grelha 9×9 é representada como uma lista plana de 81 inteiros, indexada linha a linha (da esquerda para a direita, de cima para baixo). O índice plano de uma célula na linha r e coluna c (ambas de base 1) é dado por $(r - 1) \times 9 + (c - 1)$. Uma célula vazia é representada pelo valor 0; as células preenchidas pelo puzzle (*givens*) têm valores de 1 a 9.

Esta representação plana permite calcular as coordenadas de linha, coluna e caixa sem estruturas de dados auxiliares. O índice da caixa de uma célula (r, c) é $\lfloor (r - 1)/3 \rfloor \times 3 + \lfloor (c - 1)/3 \rfloor$.

Em Prolog, os predicados fundamentais são:

- `all_different/1` — verifica que uma lista não contém duplicados;
- `valid/1` — verifica que todas as linhas, colunas e caixas satisfazem `all_different`;
- `candidates/3` — dada uma célula vazia, calcula os valores 1–9 que não violam nenhuma restrição da célula;
- `next_empty/2` — selecciona a célula vazia com menos candidatos (MRV).

Em Python (`board.py`), a representação é equivalente, com operações como `load_board/1` para ler o puzzle de ficheiro e `format_board/1` para imprimir a grelha.

2.1.2 Killer Sudoku

O Killer Sudoku partilha a mesma representação de grelha. A informação adicional das gaiolas é guardada como uma lista de pares (s, C) , onde s é a soma alvo e C é a lista de células (como pares linha-coluna de base 1) que formam a gaiola.

Em Prolog, as gaiolas são codificadas como factos `killer_cage(PuzzleId, Sum, Cells)` no ficheiro `killer_sudoku.pl`. Em Python, o formato de ficheiro é:

```
cage(Sum, [(R1,C1),(R2,C2),...]).
```

que é interpretado pela função `load_killer_puzzle` em `board.py`.

2.2 Pesquisa em Espaço de Estados — Prolog

Os algoritmos de pesquisa em Prolog operam sobre atribuições parciais: começam pelo puzzle inicial (com os givens fixos e as restantes células a zero) e atribuem progressivamente valores às células vazias. Cada nó no espaço de estados é uma grelha parcialmente preenchida que satisfaz as restrições até ao momento.

2.2.1 Aprofundamento Iterativo (DFID)

O aprofundamento iterativo com pesquisa em profundidade (DFID) é um algoritmo de pesquisa não informada que combina a completude e optimalidade da pesquisa em largura com a eficiência em memória da pesquisa em profundidade [3]. O algoritmo executa sucessivas iterações de pesquisa em profundidade limitada, aumentando o limite de profundidade em cada iteração.

Limite de Profundidade com MRV. No contexto do Sudoku, o limite de profundidade natural é o número de células vazias no puzzle: cada nó interno corresponde a uma célula preenchida, pelo que a solução está exactamente a essa profundidade. Quando se usa MRV para seleccionar a próxima célula a preencher, a pesquisa é guiada pelas células mais restringidas primeiro, o que tipicamente reduz o ramo para um único filho válido (ou um número muito pequeno). Como consequência, a iteração correspondente ao limite máximo (número de células vazias) é a única que atinge nós folha, e as iterações anteriores terminam rapidamente por não encontrar folhas. O DFID degenera efectivamente numa DFS com constraint checking, executando uma única passagem significativa.

Implementação. O predicado principal é `solve_file/2`, que lê o puzzle de um ficheiro, inicializa um contador de nós e chama `dfid/4`:

```
dfid(Board, Depth, MaxDepth, Nodes) :-
    Depth <= MaxDepth,
    ( solved(Board) -> true
    ; next_empty(Board, Pos),
      candidates(Board, Pos, Vals),
      member(Val, Vals),
      place(Board, Pos, Val, Board2),
      Nodes1 is Nodes + 1,
      dfid(Board2, Depth+1, MaxDepth, Nodes1)
    ).
```

2.2.2 Pesquisa A*

A pesquisa A* é um algoritmo de pesquisa informada que expande o nó com menor valor de $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo do caminho desde o estado inicial até n e $h(n)$ é uma estimativa heurística do custo de n até ao objectivo [3].

Heurística Admissível. A heurística adoptada é $h(n)$ = número de células vazias no estado n . Esta heurística é admissível porque, para resolver cada célula vazia, é necessário efectuar pelo menos uma atribuição, pelo que $h(n)$ nunca sobrestima o custo real. Formalmente, se a solução se encontra a k células de profundidade a partir de n , então $h(n) = k$ = custo real, pelo que $h(n) \leq h^*(n)$.

Comportamento com esta Heurística. Com $g(n)$ = número de células já preenchidas e $h(n)$ = número de células vazias, tem-se $f(n) = g(n) + h(n) = 81 - (\text{givens})$, que é constante para todos os nós ao mesmo nível de profundidade. Neste caso, o A* comporta-se essencialmente como uma pesquisa em largura ao nível da ordenação da fila de prioridade, e a poda advém inteiramente da verificação de restrições (candidatos vazios), não da ordenação heurística.

Implementação. O A* usa uma fila de prioridade implementada com a biblioteca `library(heaps)` do SWI-Prolog. Cada elemento da fila é um par (f, Board) . Uma lista fechada mantém os estados já expandidos. A expansão de cada nó usa `next_empty/2` (MRV) para seleccionar a célula a preencher:

```
astar(OpenHeap, Closed, Solution, Nodes) :-
    get_from_heap(OpenHeap, _, Board, Rest),
    ( solved(Board) -> Solution = Board
    ; \+ member(Board, Closed),
      next_empty(Board, Pos),
      candidates(Board, Pos, Vals),
      expand_children(Board, Pos, Vals, Rest,
                     [Board|Closed], NewOpen, Nodes1),
      astar(NewOpen, [Board|Closed], Solution, Nodes1)
    ).
```

2.2.3 Extensão para Killer Sudoku — Prolog

A extensão para Killer Sudoku em Prolog (`killer_sudoku.pl`) acrescenta dois predicados de verificação de gaiolas:

- `valid_cage(Board, cage(Sum, Cells))` — verifica que, quando todas as células da gaiola estão preenchidas, a sua soma é igual a `Sum` e não existem repetições;
- `partial_cage_ok(Board, cage(Sum, Cells))` — verifica a consistência parcial durante a pesquisa: se já existem valores parcialmente atribuídos na gaiola, a sua soma parcial não pode exceder `Sum`, e os valores já atribuídos não se podem repetir.

O predicado `partial_cage_ok` é crucial para a poda: ao ser chamado após cada atribuição durante a DFS/A*, elimina ramos que já violam as restrições das gaiolas, mesmo antes de a gaiola estar completamente preenchida.

Os solucionadores `solve_dfid_killer/3` e `solve_astar_killer/3` são extensões directas dos solucionadores clássicos, com a verificação adicional de `partial_cage_ok` em cada passo de expansão.

2.3 Pesquisa em Espaço de Soluções — Python

Ao contrário dos algoritmos de pesquisa em espaço de estados, o SA e o GA operam sobre soluções completas: começam com uma grelha 9×9 completamente preenchida (possivelmente inválida) e aplicam operadores que a transformam noutra solução completa, tentando minimizar o número de violações de restrições.

2.3.1 Inicialização e Operador

Inicialização. Cada caixa 3×3 é preenchida independentemente com os valores em falta (1–9 menos os givens da caixa), dispostos de forma aleatória. Esta inicialização garante que as restrições de caixa estão sempre satisfeitas por construção. Apenas as restrições de linha e coluna (e de gaiola, no Killer) podem ser violadas.

Operador de Troca Intra-caixa. O operador de vizinhança selecciona uma caixa aleatória e troca dois valores mutáveis (não-given) dentro dela. Como a troca ocorre dentro da mesma caixa, as restrições de caixa continuam satisfeitas. Formalmente, se a caixa contém os valores $\{v_1, \dots, v_9\}$ antes da troca, continua a contê-los depois.

Função de Custo. O custo $C(\text{board})$ conta as violações de linha e coluna:

$$C = \sum_{i=1}^9 (9 - |\text{distinct}(\text{row}_i)|) + \sum_{j=1}^9 (9 - |\text{distinct}(\text{col}_j)|) \quad (2.1)$$

O custo mínimo é 0 (solução completa e válida) e o máximo teórico é 144. No Killer Sudoku, soma-se adicionalmente o custo de gaiola:

$$C_{\text{cage}} = \sum_{\text{gaiola}} \mathbb{I}[\text{soma} \neq \text{alvo}] + (\text{tamanho} - |\text{distinct}(\text{gaiola})|) \quad (2.2)$$

2.3.2 Arrefecimento Simulado (SA)

O arrefecimento simulado (SA) [2] é uma metaheurística que aceita soluções piores com uma probabilidade que diminui ao longo do tempo, permitindo escapar de mínimos locais.

Esquema de Temperatura. A temperatura decresce de forma geométrica: $T_{i+1} = T_i \times \alpha$, onde $\alpha = 0.9999$ é o factor de arrefecimento. Com $T_{\text{init}} = 2.0$ e $T_{\text{min}} = 0.001$, cada execução natural tem $\approx 76\,000$ iterações.

Aceitação de Soluções. Um estado vizinho com custo C_{viz} é aceite se $C_{\text{viz}} < C_{\text{actual}}$ (melhoria) ou com probabilidade $e^{-\Delta C/T}$ se $\Delta C = C_{\text{viz}} - C_{\text{actual}} > 0$ (aceitação probabilística). Esta regra permite escapar de mínimos locais quando a temperatura é elevada.

Reinicialização por Conclusão Natural. Quando a temperatura atinge T_{min} sem encontrar a solução, o tabuleiro é reinicializado aleatoriamente e a temperatura é reposta em T_{init} . Esta estratégia de reinicialização por conclusão natural garante que cada execução percorre o ciclo completo de arrefecimento, da fase exploratória (alta temperatura) à fase explorativa (baixa temperatura), antes de recommençar. O número total de iterações é limitado a 300 000.

Mínimo Local a Custo 2. Em puzzles de elevada dificuldade, o SA pode ficar preso num estado com custo 2 — duas violações em linhas ou colunas diferentes que envolvem células em caixas distintas. O operador de troca intra-caixa não pode resolver este problema directamente: corrigir uma violação requer trocar células entre caixas diferentes, o que violaria as restrições de caixa. A baixa temperatura torna a probabilidade de aceitar estados com custo superior praticamente nula ($e^{-1/0.001} \approx 0$), pelo que o algoritmo fica bloqueado. A solução é aguardar pelo arrefecimento completo e reinicializar.

2.3.3 Algoritmo Genético (GA)

O algoritmo genético (GA) [1] mantém uma população de soluções candidatas e aplica operadores inspirados na evolução biológica (selecção, cruzamento, mutação) para melhorar a população ao longo das gerações.

Cromossoma. Cada cromossoma é uma grelha 9×9 inicializada da mesma forma que o SA (restrições de caixa garantidas por construção).

Função de Fitness. $f(\text{cromossoma}) = -C(\text{cromossoma})$, onde C é a função de custo definida na Secção 2.3.1. O GA maximiza o fitness, equivalente a minimizar o custo.

Seleccção por Torneio. São seleccionados $k = 3$ indivíduos aleatórios da população e o de maior fitness é escolhido como progenitor. Este processo é repetido para seleccionar dois progenitores (p_1, p_2) para o cruzamento.

Cruzamento Preservador de Caixas. Para cada uma das 9 caixas, o filho herda a caixa inteira do p_1 ou do p_2 com probabilidade 0.5. As células given permanecem fixas. Este operador garante que as restrições de caixa se mantêm no filho.

Mutação. Com probabilidade $p_m = 0.25$, aplica-se a um filho o operador de troca intra-caixa (igual ao do SA), realizando 2 trocas.

Elitismo e Reinicialização por Estagnação. Os 2 melhores indivíduos da geração actual sobrevivem inalterados para a próxima geração (elitismo). Se o melhor fitness não melhorar durante 2000 gerações consecutivas (estagnação), a população é reinicializada aleatoriamente, mantendo apenas o melhor indivíduo actual.

2.3.4 Extensão para Killer Sudoku — Python

A extensão para Killer Sudoku em Python é transparente para os algoritmos SA e GA: a função de custo é aumentada com o custo de gaiola C_{cage} (Secção 2.3.1), implementado em `board.py::cage_cost`. O carregamento do puzzle é tratado por `load_killer_puzzle`, que lê a grelha e as definições de gaiolas do ficheiro de texto. Tanto o SA como o GA seleccionam automaticamente o modo Killer quando o puzzle fornecido pertence ao conjunto `KILLER_PUZZLES`.

Capítulo 3

Resultados Experimentais e Conclusões

3.1 Configuração Experimental

Os testes foram realizados num MacBook Pro com processador Apple M-series, sistema operativo macOS, SWI-Prolog 10.0.2 e Python 3.14.3. Foram utilizados quatro puzzles clássicos de dificuldade crescente e um puzzle Killer Sudoku (fácil):

- **Fácil** (`classic_easy_01`) — 51 células vazias;
- **Médio** (`classic_medium_01`) — 51 células vazias;
- **Difícil** (`classic_hard_01`) — puzzle AI Escargot (17 givens);
- **Especialista** (`classic_expert_01`) — 51 células vazias;
- **Killer Fácil** (`killer_easy_01`) — 81 células vazias, 27 gaiolas.

Os solucionadores SA e GA foram executados 3 vezes por puzzle (natureza estocástica). O limite de tempo de Prolog foi de 120 segundos por chamada. O SA usou no máximo 300 000 iterações. O GA usou população de 200 indivíduos e no máximo 10 000 gerações.

3.2 Resultados dos Solucionadores Prolog

A Tabela 3.1 apresenta os resultados do DFID e A* para os puzzles clássicos.

Tabela 3.1: Resultados do DFID e A* nos puzzles clássicos.

Puzzle	DFID		A*	
	Nós	Tempo (s)	Nós	Tempo (s)
Fácil	51	0.163	51	0.173
Médio	49	0.153	49	0.160
Difícil	481	2.671	2276	13.314
Especialista	846	3.667	216	1.024

Análise DFID. O DFID com MRV expande poucos nós em todos os puzzles. O MRV reduz drasticamente o factor de ramificação ao escolher sempre a célula com menos candidatos válidos. Em puzzles bem constrangidos (fácil, médio), o número de nós expandidos é inferior a 55, o que confirma que a pesquisa é efectivamente uma DFS com verificação de restrições que raramente precisa de retroceder. No puzzle Difícil (AI Escargot, com apenas 17 givens), o DFID expande 481 nós — o maior esforço de pesquisa entre os casos testados com este algoritmo.

Análise A*. O A* expande exactamente o mesmo número de nós que o DFID nos puzzles Fácil e Médio. No puzzle Difícil, o A* expande 2276 nós (vs. 481 do DFID), porque a heurística h = células vazias é constante ao mesmo nível de profundidade, fazendo o A* comportar-se como BFS. A fila de prioridade cresce rapidamente, aumentando o overhead por nó. No puzzle Especialista, o A* expande apenas 216 nós vs. 846 do DFID, possivelmente devido a diferenças na ordenação da fila de prioridade em casos de empate de prioridade.

Resultados Killer Sudoku — Prolog. A Tabela 3.2 mostra os resultados para o puzzle Killer Fácil.

Tabela 3.2: Resultados do DFID e A* no Killer Sudoku (fácil).

Algoritmo	Nós Expandidos	Tempo (s)
DFID Killer	51	0.188
A* Killer	51	0.176

O puzzle Killer Fácil (sem givens iniciais) é resolvido com apenas 51 nós por ambos os algoritmos. A poda por `partial_cage_ok` é muito eficaz: as restrições de gaiola eliminam candidatos logo nas primeiras atribuições, reduzindo o espaço de busca de forma semelhante aos givens de um puzzle clássico.

3.3 Resultados do Arrefecimento Simulado (SA)

A Tabela 3.3 apresenta os resultados do SA nas 3 execuções por puzzle.

Tabela 3.3: Resultados do SA (3 execuções por puzzle; limite: 300 000 iterações).

Puzzle	Resultado	Execuções Resolvidas	Iter. (média)	Tempo (s)
Fácil	Resolvido	3/3	≈17 644	0.06–0.10
Médio	Resolvido	3/3	≈44 223	0.08–0.38
Difícil	Não resolvido	0/3	300 000	≈1.21
Especialista	Parcial	1/3	≈95 647	0.38
Killer Fácil	Resolvido	3/3	≈119 560	0.87–1.58

O SA resolve de forma fiável os puzzles Fácil e Médio em poucas dezenas de milissegundos. O puzzle Difícil (AI Escargot) constitui o caso mais desafiante: em todas as execuções, o algoritmo fica bloqueado num estado com custo 2, que corresponde a duas violações residuais em linhas/colunas distintas, cujas células envolvidas pertencem a caixas diferentes. O operador de troca intra-caixa não consegue resolver este conflito sem aceitar temporariamente estados de custo superior, o que a temperatura baixa torna improvável. O puzzle Especialista é resolvido em 1 de cada 3 execuções, demonstrando a variabilidade estocástica do algoritmo.

O puzzle Killer Fácil é resolvido em todas as execuções, apesar de não ter givens iniciais, porque as restrições de gaiola reduzem substancialmente o espaço efectivo de soluções.

3.4 Resultados do Algoritmo Genético (GA)

A Tabela 3.4 apresenta os resultados do GA.

Tabela 3.4: Resultados do GA (3 execuções; pop.=200, máx. 10 000 gerações, 15–26 s/execução).

Puzzle	Resultado	Execuções Resolvidas	Custo Mínimo
Fácil	Não resolvido	0/3	6
Médio	Não resolvido	0/3	2
Difícil	Não resolvido	0/3	4
Especialista	Não resolvido	0/3	4
Killer Fácil	Não resolvido	0/3	15

O GA não encontrou uma solução completa em nenhuma das execuções realizadas. O principal problema é a convergência prematura: a diversidade genética da população colapsa rapidamente, e o GA queda num mínimo local sem conseguir sair, mesmo com a reinicialização por estagnação. O cruzamento preservador de caixas, apesar de manter as restrições de caixa, não introduz diversidade suficiente para explorar espaços de soluções de dimensão elevada.

O custo mínimo atingido pelo GA é consistentemente superior ao do SA nos mesmos puzzles. No puzzle Killer Fácil, o custo mínimo de 15 reflecte que o GA raramente satisfaz as restrições de gaiola, que exigem uma combinação muito específica de valores.

3.5 Análise Comparativa

Tabela 3.5: Comparação dos quatro solucionadores nos puzzles clássicos.

Puzzle	DFID	A*	SA	GA
Fácil	✓ 0.16s	✓ 0.17s	✓ 0.06–0.10s	× custo 6
Médio	✓ 0.15s	✓ 0.16s	✓ 0.08–0.38s	× custo 2
Difícil	✓ 2.67s	✓ 13.3s	× custo 2	× custo 4
Especialista	✓ 3.67s	✓ 1.02s	1/3 resolvido	× custo 4

Os algoritmos de pesquisa em espaço de estados (DFID, A*) são completos e óptimos: se existir uma solução, encontram-na sempre. Os algoritmos de optimização (SA, GA) são incompletos: podem não encontrar a solução mesmo que ela exista. Em contrapartida, o SA tem um desempenho muito bom em puzzles fáceis e médios, com tempos inferiores aos do Prolog (overhead de interpretação).

O A* é menos eficiente que o DFID no puzzle Difícil porque a heurística adoptada não distingue estados ao mesmo nível de profundidade, fazendo a fila de prioridade crescer excessivamente. O DFID, com a sua natureza DFS, mantém um número constante de nós em memória, o que o torna mais adequado para puzzles muito restringidos.

3.6 Conclusões

Este trabalho implementou e comparou quatro abordagens algorítmicas para a resolução do Sudoku clássico e do Killer Sudoku. As principais conclusões são:

1. **DFID com MRV** é o solucionador mais robusto e fiável: resolve todos os puzzles testados, com uso mínimo de memória e tempos razoáveis. A heurística MRV reduz o espaço de busca de forma tão eficaz que o DFID degenera numa DFS com verificação de restrições.
2. **A*** partilha a completude do DFID, mas a heurística h = células vazias não distingue estados ao mesmo nível, tornando-o equivalente a uma pesquisa em largura guiada por restrições. Em puzzles com muitos estados alternativos, o A* expande significativamente mais nós que o DFID.
3. **SA** é eficaz para puzzles de dificuldade baixa a média, com tempos muito competitivos. A sua principal limitação é o bloqueio em mínimos locais de custo 2, que o operador intra-caixa não consegue resolver directamente a baixa temperatura. A extensão para Killer Sudoku é elegante: basta adicionar o custo de gaiola à função de custo.
4. **GA** sofre de convergência prematura e não encontrou soluções completas nos testes realizados. O operador de cruzamento preservador de caixas mantém a validade das restrições de caixa, mas não diversifica suficientemente a população para escapar de mínimos locais profundos.
5. A extensão para **Killer Sudoku** foi implementada com sucesso em ambas as linguagens. Em Prolog, a poda por `partial_cage_ok` é muito eficaz, reduzindo o espaço de busca mesmo sem givens iniciais. Em Python, a adição do custo de gaiola à função de custo é directa e transparente para os algoritmos SA e GA.

Como trabalho futuro, seria interessante explorar operadores inter-caixa controlados (que preservem as restrições de caixa mas permitam mais mobilidade), técnicas de reheating no SA para escapar de mínimos locais profundos, e operadores de cruzamento mais diversificados no GA (por exemplo, cruzamento a nível de linha/coluna em vez de caixa).

Referências

- [1] John H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [2] Scott Kirkpatrick, C. Daniel Gelatt & Mario P. Vecchi, “Optimization by Simulated Annealing”, *Science*, vol. 220, n.º 4598, páginas 671–680, 1983.
- [3] Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Pearson, 2020.
- [4] Takayuki Yato & Takahiro Seta, “Complexity and Completeness of Finding Another Solution and its Application to Puzzles”, em *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E86-A, 2003, páginas 1052–1060.

Apêndice A

Estrutura do Repositório e Instruções de Execução

A.1 Estrutura de Ficheiros

O repositório está organizado da seguinte forma:

```
ai-killer-sudoku/  
  src/  
    prolog/  
      sudoku.pl          - Predicados base do Sudoku classico  
      dfid.pl            - Solucionador DFID com MRV  
      astar.pl           - Solucionador A* (h=celulas vazias)  
      killer_sudoku.pl   - Extensao Killer Sudoku (DFID + A*)  
    python/  
      board.py           - Representacao da grelha e utilitarios  
      puzzles.py          - Caminhos dos puzzles  
      sa_solver.py        - Solucionador SA  
      ga_solver.py        - Solucionador GA  
  data/  
    input/  
      classic_easy_01.txt  
      classic_medium_01.txt  
      classic_hard_01.txt  
      classic_expert_01.txt  
      killer_easy_01.txt  
    output/  
      benchmark_results.txt  
      sa_results.txt  
      ga_results.txt  
  scripts/  
    benchmark.sh          - Script de benchmark completo
```

A.2 Instruções de Execução

Requisitos. SWI-Prolog (swipl) e Python 3.10+ devem estar instalados e disponíveis no PATH. Não são necessárias bibliotecas Python externas.

Benchmark Completo. Para executar todos os solucionadores em todos os puzzles e gerar o relatório de resultados:

```
bash scripts/benchmark.sh
```

Solucionadores Prolog individualmente.

```
# DFID -- puzzle fácil
swipl -q -s src/prolog/dfid.pl \
    -g "solve_file('data/input/classic_easy_01.txt', _), halt"

# A* -- puzzle especialista
swipl -q -s src/prolog/astar.pl \
    -g "solve_file('data/input/classic_expert_01.txt', _), halt"

# Killer Sudoku (DFID + A*)
swipl -q -s src/prolog/killer_sudoku.pl \
    -g "solve_killer_file(easy_01, _), halt"
```

Solucionadores Python individualmente.

```
# SA -- puzzle médio, 5 execuções
python3 src/python/sa_solver.py --puzzle medium --runs 5

# GA -- puzzle fácil, população 300, 5000 gerações
python3 src/python/ga_solver.py --puzzle easy --pop 300 \
    --generations 5000 --runs 3

# SA -- Killer Sudoku
python3 src/python/sa_solver.py --puzzle killer --runs 3

# GA -- Killer Sudoku
python3 src/python/ga_solver.py --puzzle killer --pop 200 \
    --generations 10000 --runs 3
```